DE MONTFORT UNIVERSITY CAMBODIA

Operating Systems & Networks
CTEC1704C

Lecture – 8 : Permissions Processes and Memory

Dr. Sameer Jha, SFHEA
Faculty of Computing
De Montfort University Cambodia

DE MONTFORT UNIVERSITY CAMBODIA

Outline

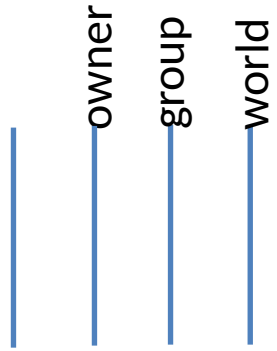
- Permissions
- Access Control Lists on Linux
- Process Management
- Process Control Block (PCB)
- Virtual Memory

Permissions

- File owner/creator should be able to control:
  - what can be done
  - by whom
- This information is kept in the Access Control List (ACL) for each file and directory
- Types of access: read (r), write (w), execute (x)
- Three classes of users:
  - Owner: user who created the file
  - Group: group of users who have shared access
  - World: everybody else

Access Control Lists on Linux

- Permissions are maintained for each class of user:



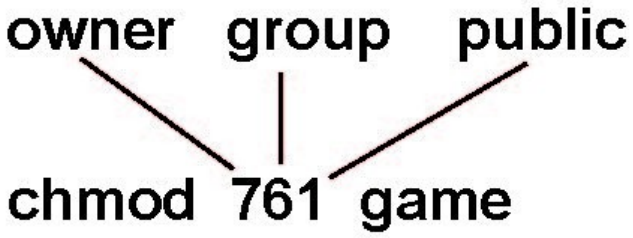

```
-rw-r--r--   1 ali   staff   4.9K   6 Oct 11:44 archive-file.zip
drwxr-xr-x  46 ali   staff   1.5K   8 Oct 14:28 architecture
-rwxr-xr-x   1 ali   staff    71K 26 Oct 18:57 schedule.ods
```

- Alternate way of writing *rwx*: assign numbers
  - Read = 4, Write = 2, Execute = 1
- Add numbers for each class of user

		r w x	
a) owner access	7	⇒	1 1 1
b) group access	6	⇒	1 1 0
c) public access		1	⇒ 0 0 1

Modifying Access Control

For a particular file (say *game*) or subdirectory, define appropriate access using *chmod*:



Equivalent:

chmod u+rwx game
chmod g+rw game
chmod o+x game

Modifying a file's group:

chgrp groupname filename

Adding users to groups:

usermod –a –G groupname username

What is a process?

Program is **passive entity** stored on disk (executable file)

Process is **active entity**: a program in execution

- Program becomes process when executable file loaded into memory

Process execution must progress in sequential fashion

One program can be several processes

- Consider multiple users executing the same program

Process Management

Process needs resources to accomplish its task

- CPU, memory, I/O, files
- Initialization data

- Process termination requires reclaim of any reusable resources
- Operating system is responsible for
  - Creating and deleting both user and system processes
  - Suspending and resuming processes

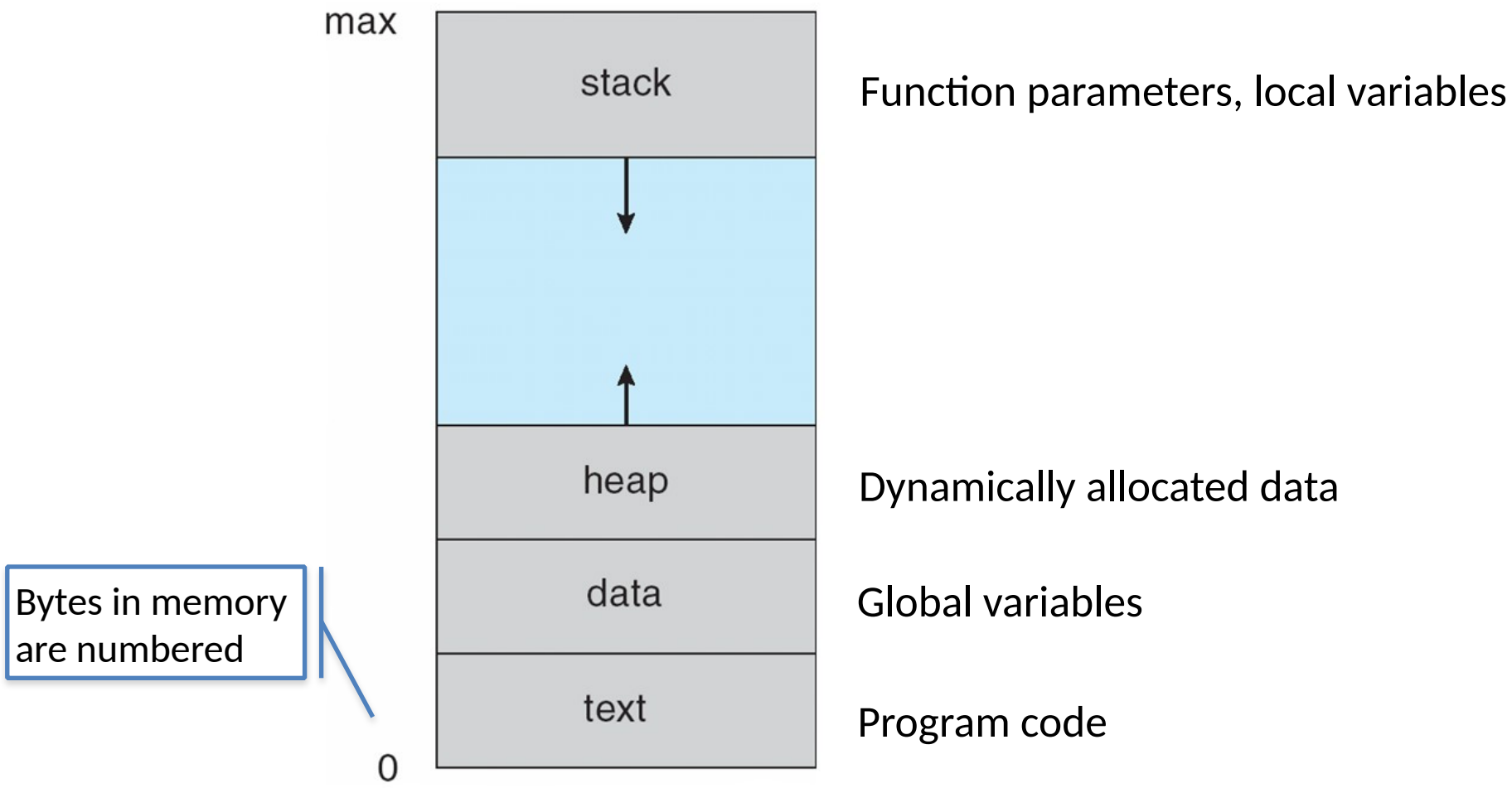
Parts of a Process

- The program code, also called **text section**
- Current activity
  - program counter
  - processor registers

Learnt these in the architecture lectures!

- **Stack** containing temporary data
  - Function parameters, return addresses, local variables
- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time

Process in Memory



Process State

As a process executes, it changes state

new: The process is being created

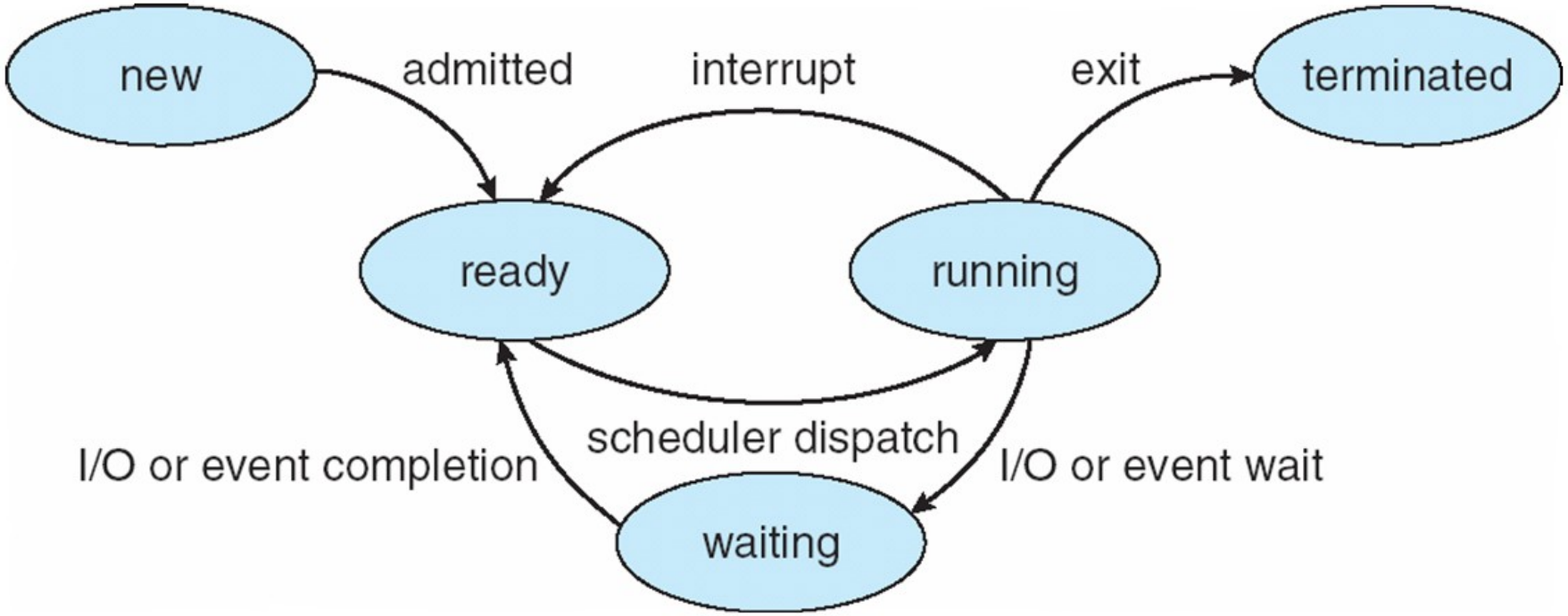
running: Instructions are being executed

waiting: The process is waiting for some event to occur

ready: The process is waiting to be assigned to a processor

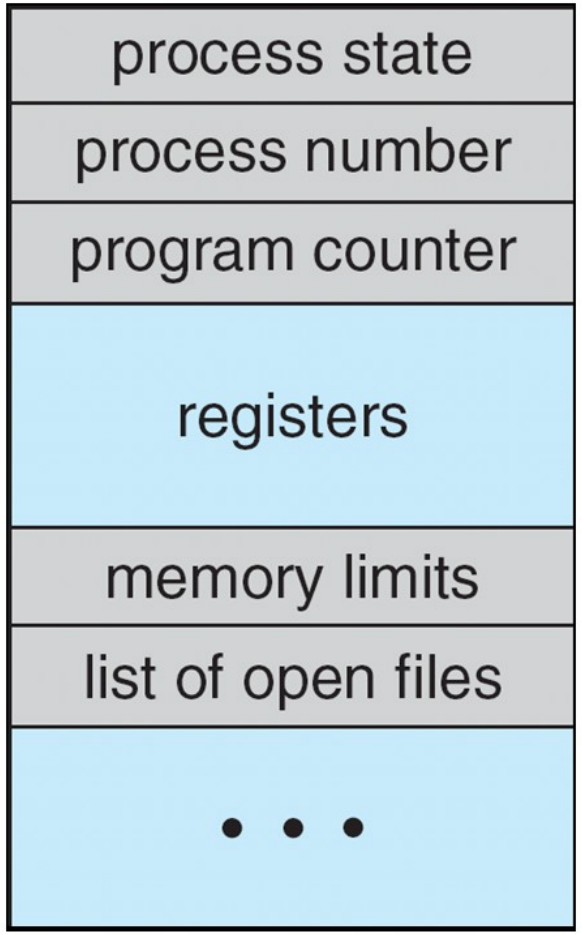
terminated: The process has finished execution

Diagram of Process State



Process Control Block (PCB)

Information associated with each process (also called task control block)
Process state – running, waiting, etc.
Program counter – location of instruction to execute next
CPU registers – contents of all process-centric registers
CPU scheduling information – priorities, scheduling queue pointers
Memory-management information – memory allocated to the process
Accounting information – CPU used, clock time elapsed since start, time limits
I/O status information – I/O devices allocated to process, list of open files



Where to find the PCB in Linux?

Look in /proc …
/proc/X/status
/proc/X/sched
/proc/X/maps
/proc/X/stack
Where X is the process id

Look in `/proc` …

- "`/proc/X/status`"
- "`/proc/X/sched`"
- "`/proc/X/maps`"
- "`/proc/X/stack`

  Where `X` is the process ID"

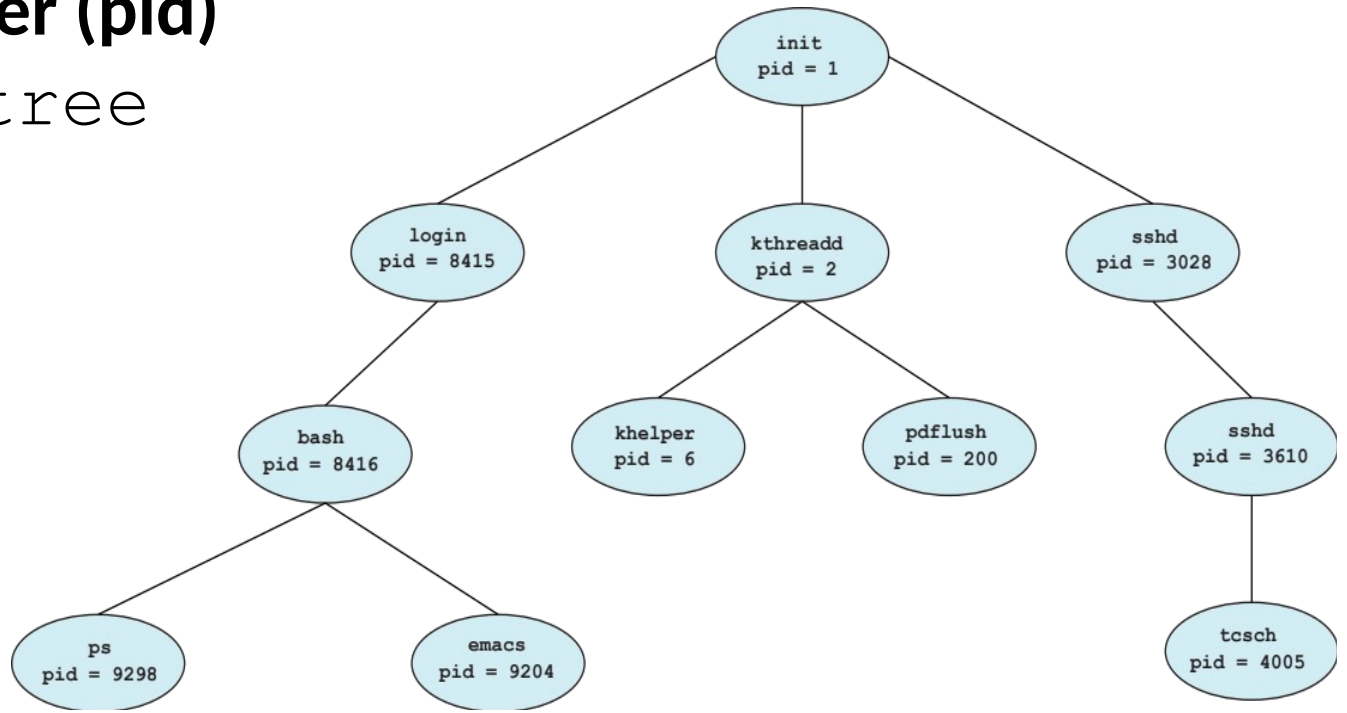
```
isabel@dmu023694 ~ $ cat /proc/1/status
Name:   init
State:  S (sleeping)
Tgid:   1
Ngid:   0
Pid:    1
PPid:   0
TracerPid:      0
Uid:    0       0       0       0
Gid:    0       0       0       0
FDSize: 64
Groups:
VmPeak:    15532 kB
VmSize:    15496 kB
VmLck:         0 kB
VmPin:         0 kB
VmHWM:      1888 kB
VmRSS:      1888 kB
VmData:      224 kB
VmStk:       132 kB
VmExe:        36 kB
VmLib:      2744 kB
VmPTE:        60 kB
VmSwap:        0 kB
Threads:        1
SigQ:   0/95653
SigPnd: 0000000000000000
ShdPnd: 0000000000000000
SigBlk: 0000000000000000
SigIgn: fffffffe57f0d8fc
SigCgt: 00000001a80b2603
CapInh: 0000000000000000
CapPrm: 000000ffffffffff
CapEff: 000000ffffffffff
CapBnd: 000000ffffffffff
Seccomp:        0
Speculation_Store_Bypass:       vulnerable
Cpus_allowed:   ffff,ffffffff,ffffffff,ffffffff,ffffffff
Cpus_allowed_list:      0-143
Mems_allowed:   00000000,00000001
Mems_allowed_list:      0
voluntary_ctxt_switches:        56302
nonvoluntary_ctxt_switches:     130
```

Context Switch

- When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process via a **context switch**
- **Context** of a process represented in the **PCB**
- Context-switch time is overhead; the system does no useful work while switching
  - The more complex the OS and the PCB -> the longer the context switch
- Time dependent on hardware support
  - Some hardware provides multiple sets of registers per CPU -> multiple contexts can be loaded at once

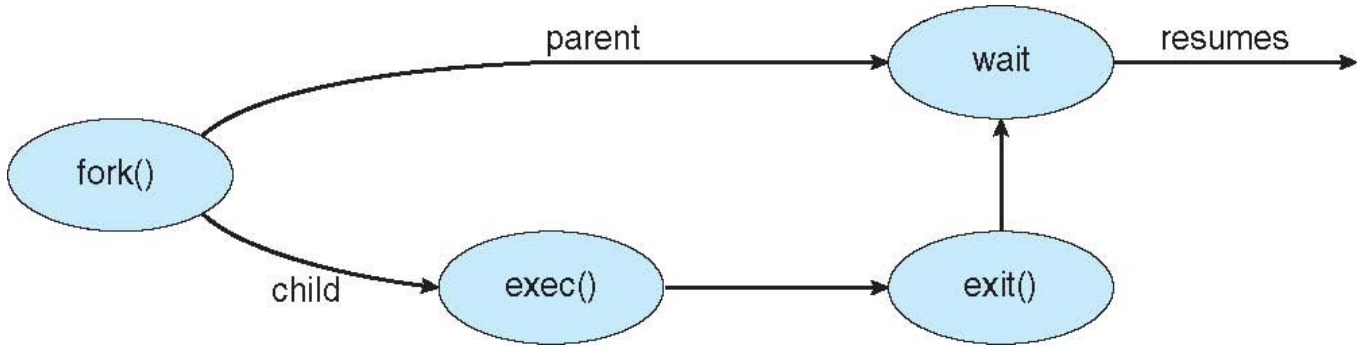
Process Creation

Parent process create **children** processes, which, in turn create other processes, forming a **tree** of processes
Generally, process identified and managed via a **process identifier (pid)**
Try `pstree`



Process Creation (Cont.)

- **fork()** system call creates new process, child is exact duplicate of parent
- **exec()** system call used after a fork() to replace the process' memory space with a new program
- **wait()** system call can be used by parent to wait for termination of a child process
- **exit()** system call terminates the process



Process Termination

Process executes last statement and then asks the operating system to delete it using **exit()**

- Returns status data from child to parent (via **wait()**)
- Process' resources are deallocated by operating system

Parent may terminate the execution of children processes using the **abort()** system call. Some reasons for doing so:

- Child has exceeded allocated resources
- Task assigned to child is no longer required
- The parent is exiting

When process terminates and no parent is waiting (has not (yet) invoked wait()), process is a **zombie**

When parent process terminates without invoking wait, process is an **orphan**

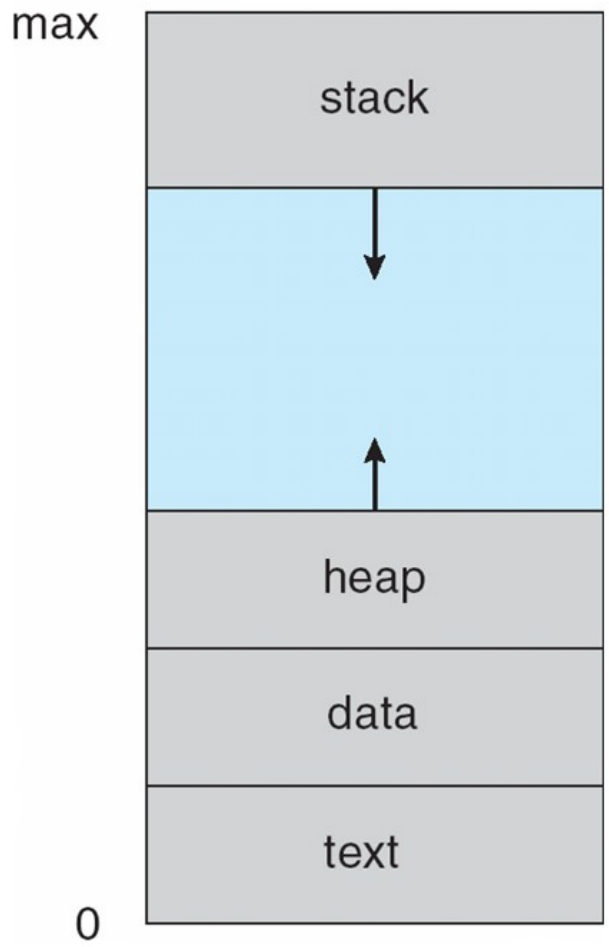
Recall: Parts of a Process in Memory

So far, we've studied the memory layout for individual processes

- Each process uses addresses from 0 to max

The operating system has to support many processes running at the same time

How to organize the memory spaces for all processes, while making sure that memory is used efficiently?

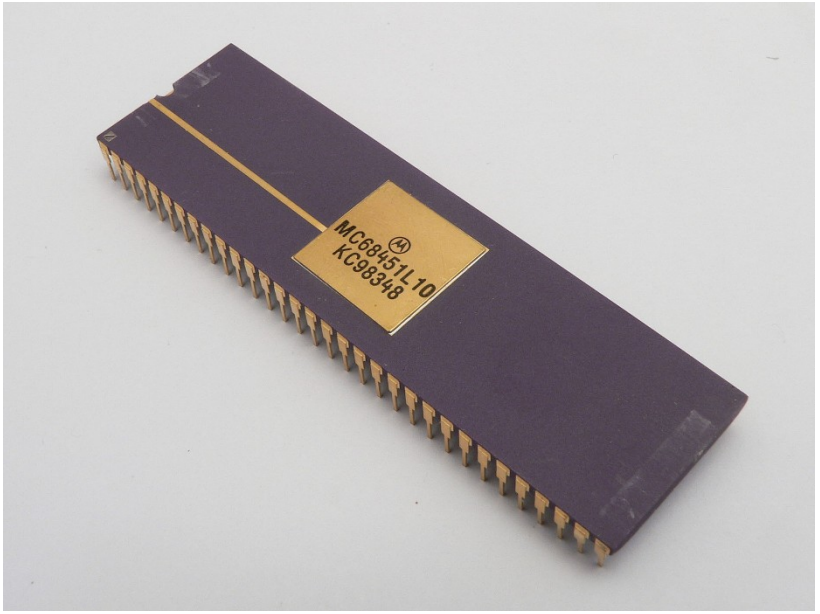


Address Spaces

- Address space = set of all addresses
- The operating system supports two address spaces:
  - **Logical** addresses (also called **virtual** addresses), generated by the CPU and used by processes
  - **Physical** addresses, used by the memory management unit
- The memory management unit translates logical addresses into physical addresses
- The concept of a logical address space that is bound to a separate physical address space is central to proper memory management

Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address
- The user program deals with logical addresses; it never sees the real physical addresses
  - Execution-time binding occurs when reference is made to location in memory
  - Logical address bound to physical addresses



Source: David Monniaux, https://com[illegible]wiki/File:MC68451_p1160081.jpg

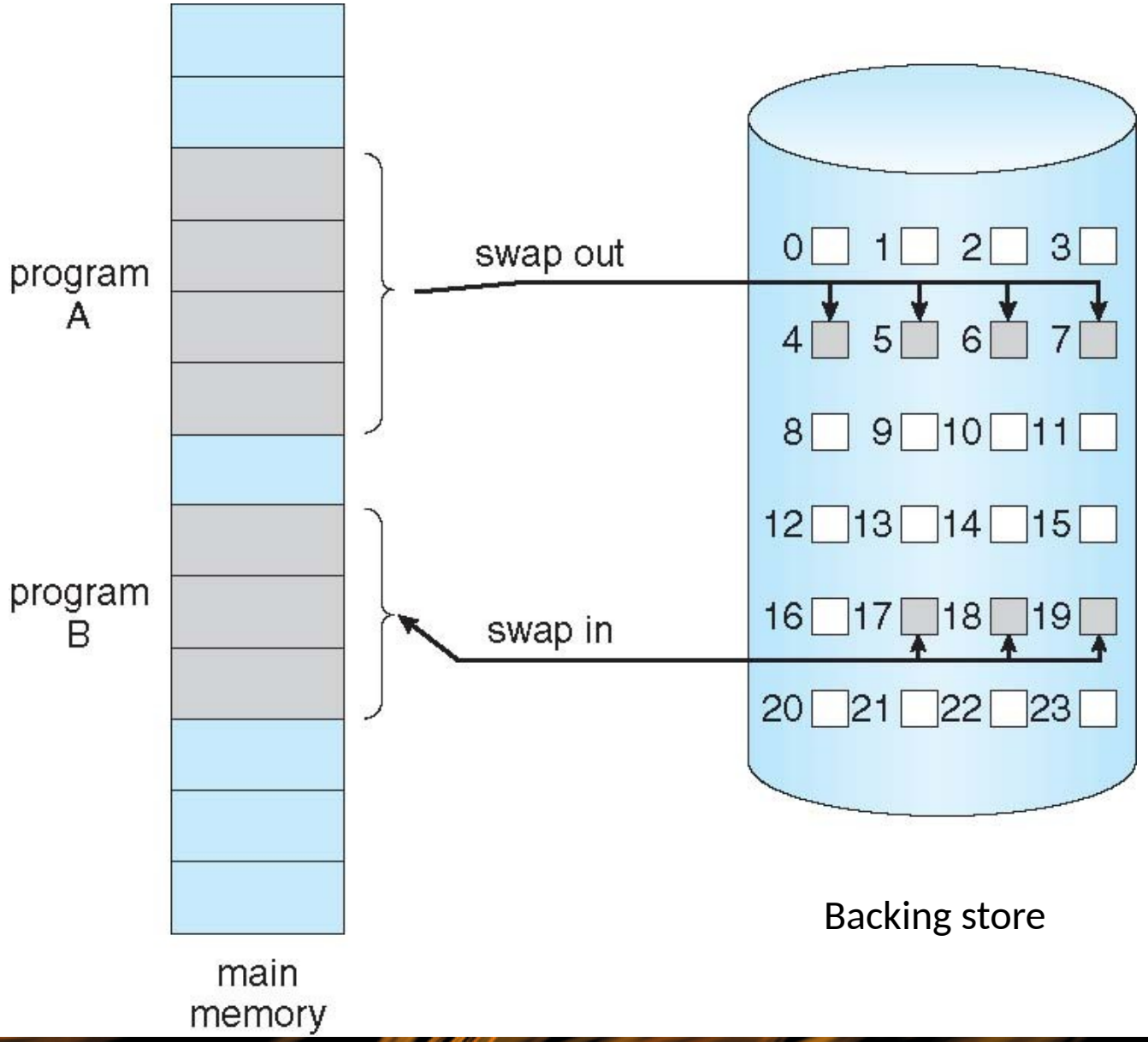
Few things to consider

- Code needs to be in memory to execute, but **entire program rarely used**
  - E.g., error code, unusual routines, large data structures
  - ➢ Entire program code not needed at same time
- Consider ability to **execute partially-loaded program**
  - Program no longer constrained by limits of physical memory
  - Each program takes less memory while running -> more programs run at the same time
    - Increased CPU utilization and throughput with no increase in response time or turnaround time
  - Less I/O needed to load or swap programs into memory -> each user program runs faster

Swapping

- A process can be **swapped** temporarily out of memory to a backing store, and then brought back into memory for continued execution
  - Total physical memory space of processes can exceed physical memory
- **Backing store:** fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- Major part of swap time is transfer time
- Total transfer time is directly proportional to the amount of memory swapped

Schematic View of Swapping



Context Switch Time and Swapping

- If next processes to be put on CPU is not in memory, need to swap out a process and swap in target process
- Context switch time can then be very high
- 100MB process swapping to hard disk with transfer rate of 50MB/sec
  - Swap out time of 2000 ms
  - Plus swap in of same sized process
  - Total context switch swapping component time of 4000ms (4 seconds)
  - Compare with ~10us for context switch without swapping!

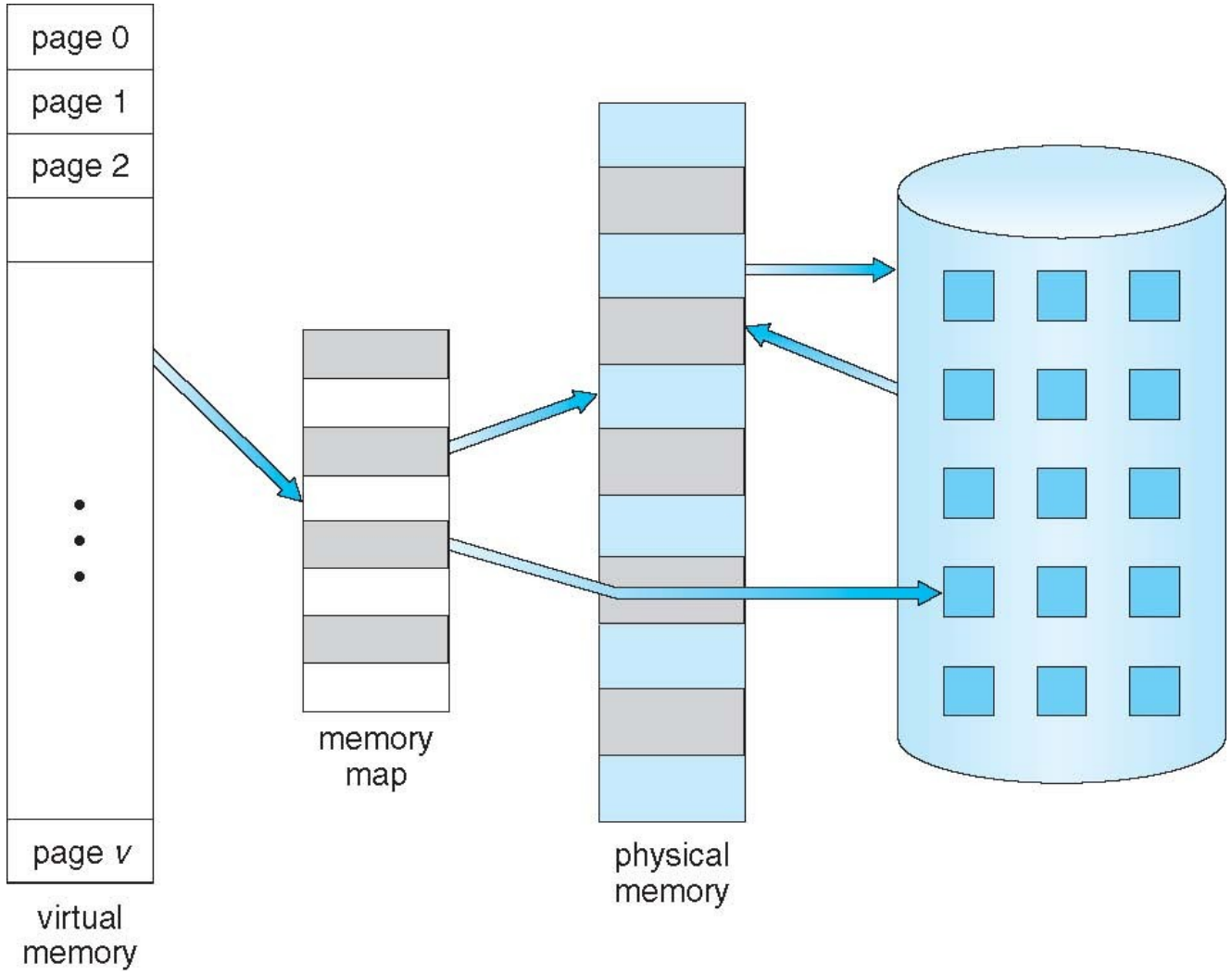
Virtual Memory

- Virtual memory separates logical memory from physical memory
  - Only **part of the program** needs to be in memory for execution
  - Logical address space can therefore be **much larger** than physical address space
  - Allows address spaces to be **shared** by several processes
  - Allows for more efficient process creation
  - More programs running concurrently
  - Less I/O needed to load or swap processes

Virtual Address Space

- Virtual address space = logical view of how process is stored in memory
  - Usually start at address 0, contiguous addresses until end of space
  - Meanwhile, physical memory organized in frames
  - MMU must map logical to physical
- Virtual memory can be implemented via:
  - **Demand paging**
  - Demand segmentation

Virtual Memory That is Larger Than Physical Memory



Virtual-address Space

Logical address space: stack grows “down” while heap grows “up”

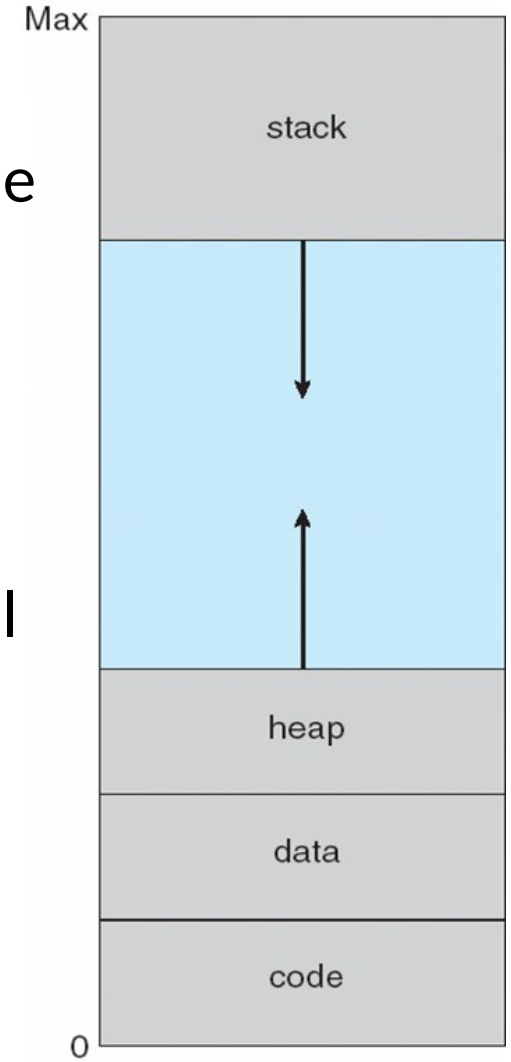
- Unused address space between the two is hole, no physical memory needed until heap or stack grows to a new page

Enables **sparse address spaces** with holes for growth, dynamically linked libraries, etc.

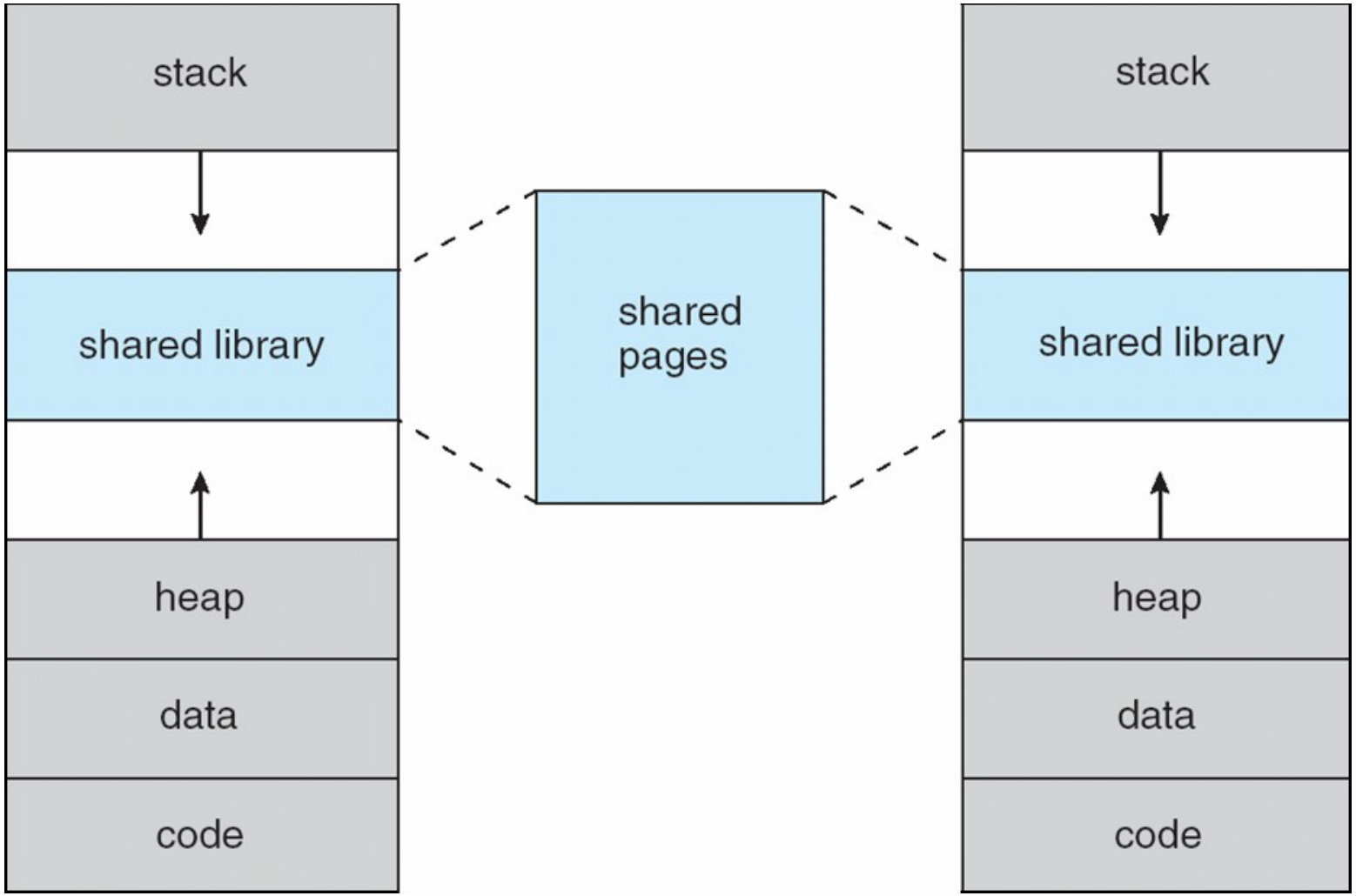
System libraries shared via mapping into virtual address space

Shared memory by mapping pages read-write into virtual address space

Pages can be **shared during fork()**, speeding process creation



Shared Library Using Virtual Memory



DE MONTFORT UNIVERSITY CAMBODIA

DE MONTFORT UNIVERSITY CAMBODIA

Thank You!